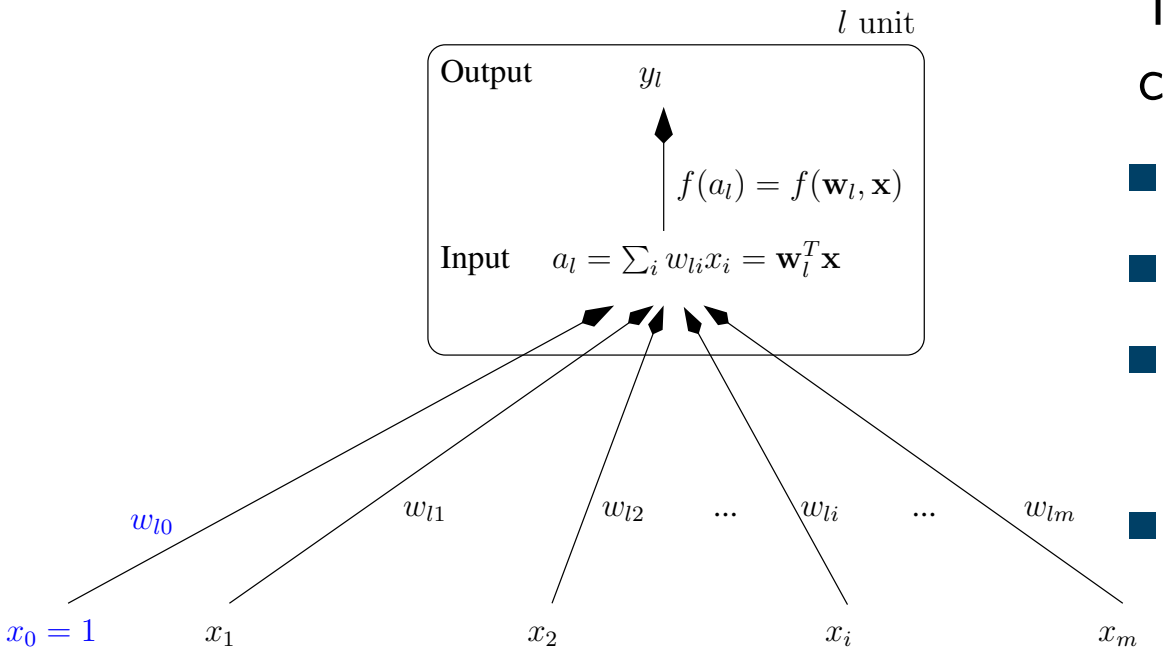
Multi-layer perceptrons

How to think/work with MLPs

Two ways to approach MLPs:

- the single unit view: operations are described in the unit level relying on inner products.
- the layer view: operations are described on the layer (=sets of units of the given layer) level using matrix operations.

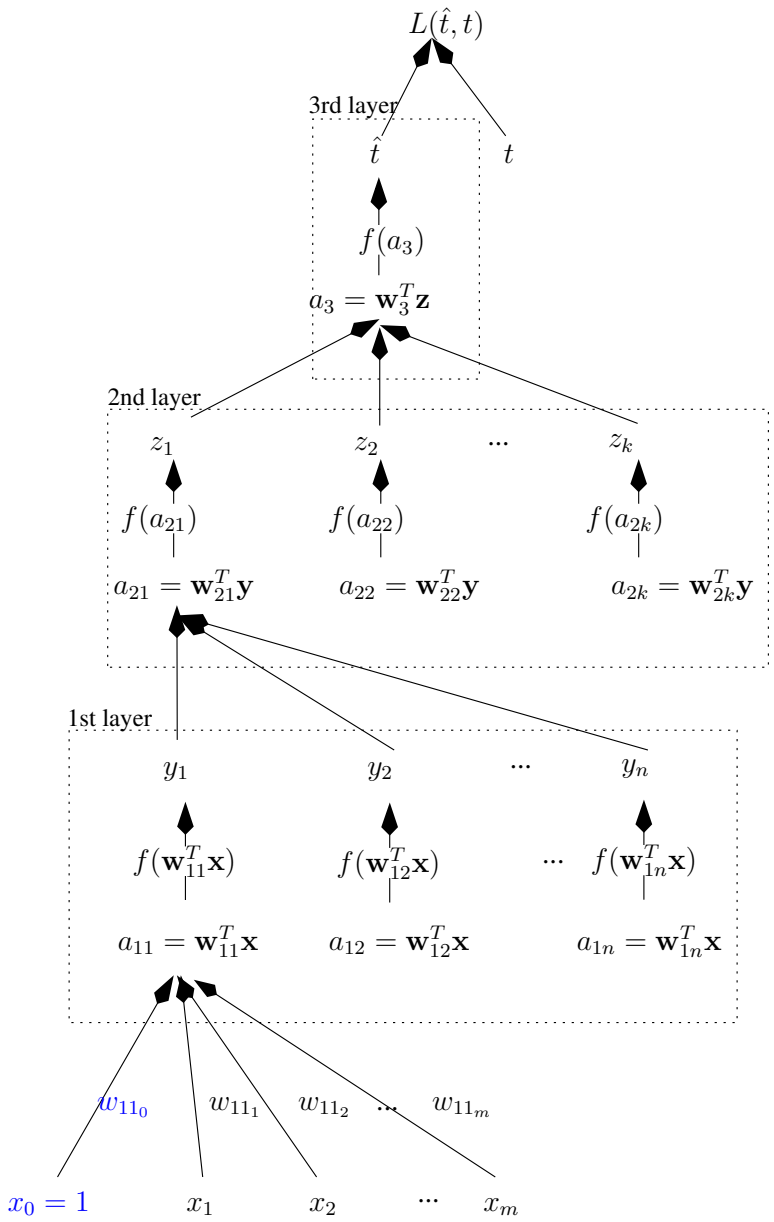
The single unit view



The l unit is characterised/described by:

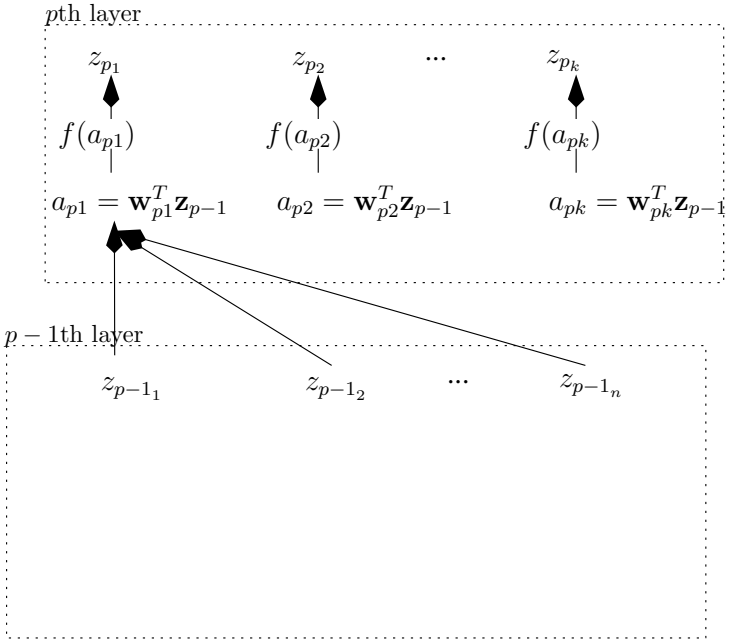
- Its *input* vector $\mathbf{x}$
- Its *parameter* vector $\mathbf{w}_l$
- The *non-linear* function $f()$ that generates its output
- Its output y_l

Three Layer Network, detailed



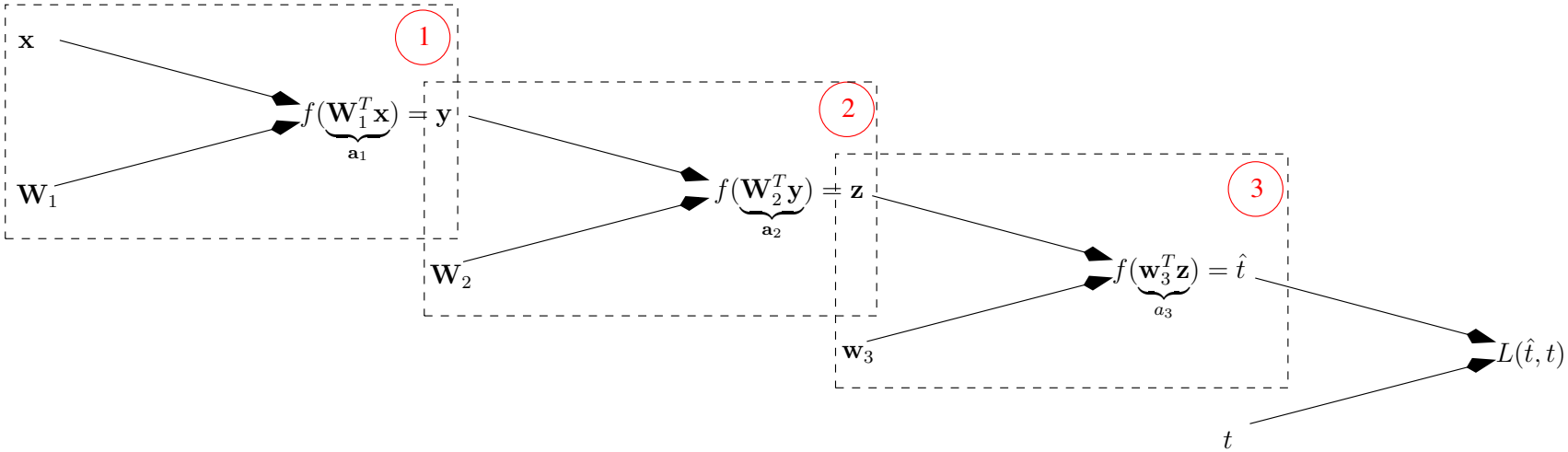
	Loss layer
Output :	$L(\hat{t}, t)$
Input :	$\hat{t}, t$
	3rd layer/last
Output :	$\hat{t} \in R$
I2O func.	$\hat{t} = f(a_3) = f(\mathbf{w}_3^T \mathbf{z})$
Activation:	$a_3 \in R$
	$a_3 = \mathbf{w}_3^T \mathbf{z}$
Params:	$\mathbf{w}_3 = (\mathbf{w}_{31}, ..., \mathbf{w}_{3k})^T \in R^k$
Input :	$\mathbf{z} \in R^k$
	2nd layer
Output :	$\mathbf{z} = (z_1, \ldots, z_k)^T \in R^k$
I2O func.	$\mathbf{z} = f(\mathbf{a}_2) = f(\mathbf{W}_2^T \mathbf{y})$
Activation:	$\mathbf{a}_2 = (a_{21}, a_{22}, ...a_{2k})^T \in R^k$
	$\mathbf{a}_2 = \mathbf{W}_2^T \mathbf{y}$
Params:	$\mathbf{W}_2 = [\mathbf{w}_{21}, ..., \mathbf{w}_{2n}] \in R^{n \times k}$
Input :	$\mathbf{y} \in R^n$
	1st layer
Output :	$\mathbf{y} = (y_1, \ldots, y_n)^T \in R^n$
I2O func.	$\mathbf{y} = f(\mathbf{a}_1) = f(\mathbf{W}_1^T \mathbf{x})$
Activation:	$\mathbf{a}_1 = (a_{11}, a_{12}, ...a_{1n})^T \in R^n$
	$\mathbf{a}_1 = \mathbf{W}_1^T \mathbf{x}$
Params :	$\mathbf{W}_1 = [\mathbf{w}_{11}, ..., \mathbf{w}_{1n}] \in R^{m \times n}$
Input :	$\mathbf{x} \in R^m$

Generalised layer form, pth layer

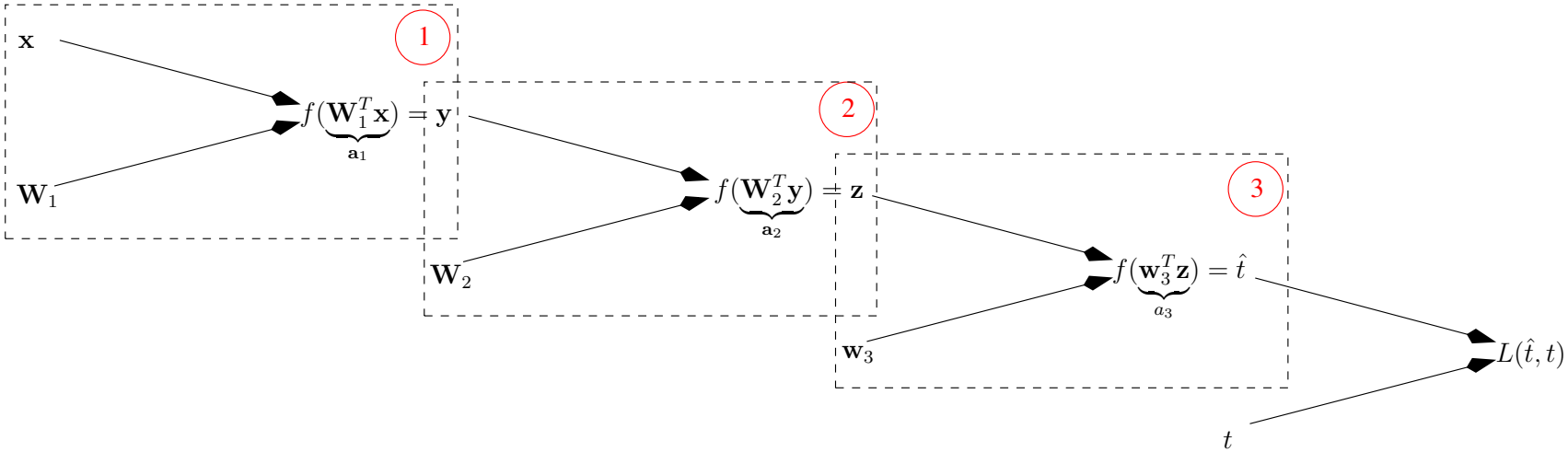


	pth layer
Output :	$\mathbf{z}_p = (z_{p_1}, \ldots, z_{p_k})^T \in R^k$
I2O func.	$\mathbf{z}_p = f(\mathbf{a}_p) = f(\mathbf{W}_p^T \mathbf{z}_{p-1})$
Activation:	$\mathbf{a}_p = (a_{p_1}, a_{p_2}, ...a_{p_k})^T \in R^k$
	$\mathbf{a}_p = \mathbf{W}_p^T \mathbf{z}_{p-1}$
Params :	$\mathbf{W}_p = [\mathbf{w}_{p1}, ..., \mathbf{w}_{pk}] \in R^{n \times k}$
Input :	$\mathbf{z}_{p-1} = (z_{p-1_1}, ..., z_{p-1_n})^T \in R^n$

Three Layer Network, simplified

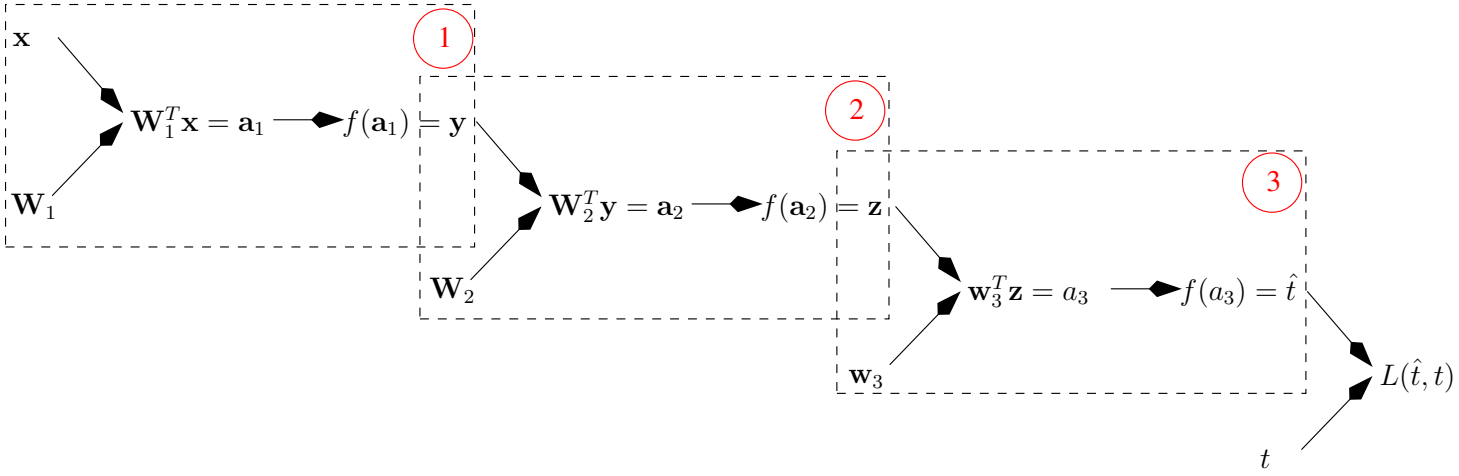


Three Layer Network, simplified and its Jacobians



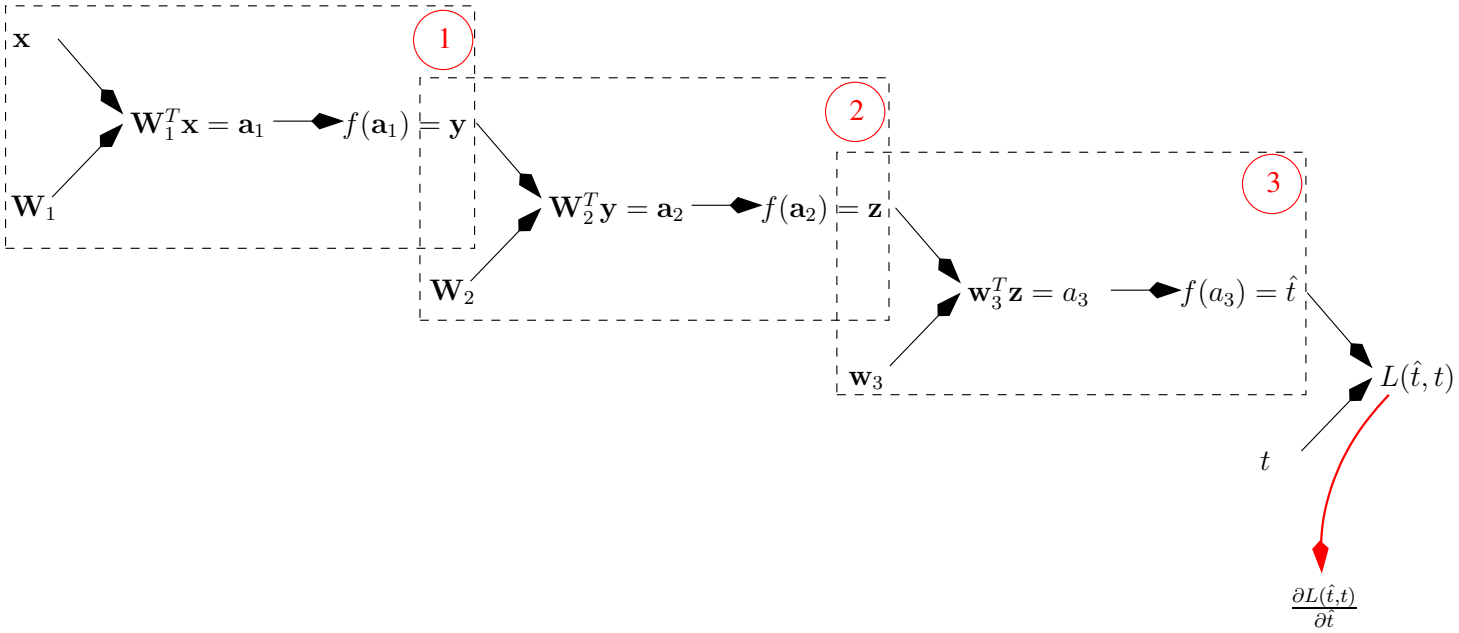
Three Layer Network, simplified and its Jacobians

- Networks are computational graphs
- Each node in such a graph performs an operation:
  - typical computations in *forward propagation*: matrix multiplications, elementwise non-linearities
  - typical computations in *back propagation*: gradients and Jacobians



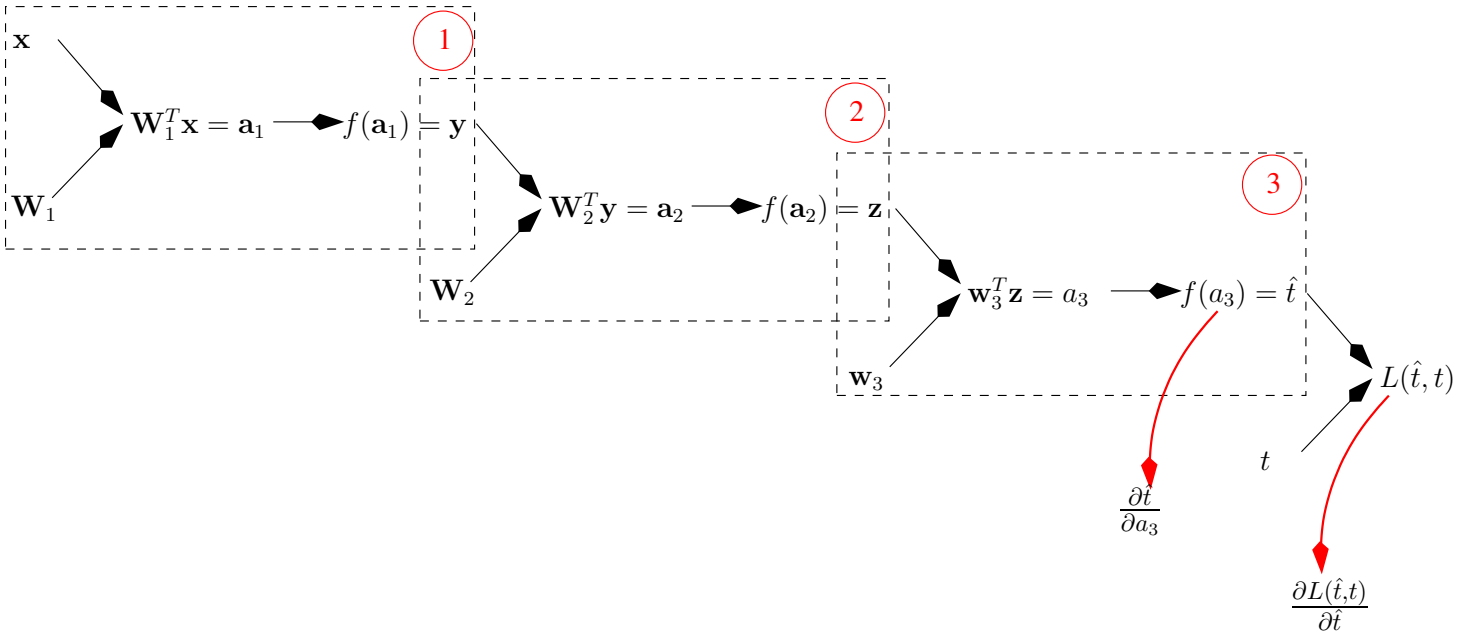
Three Layer Network, simplified and its Jacobians

- Each node should know how to calculate its derivative, gradient, or Jacobian, with respect to:
  - its *input*
  - its *parameters*



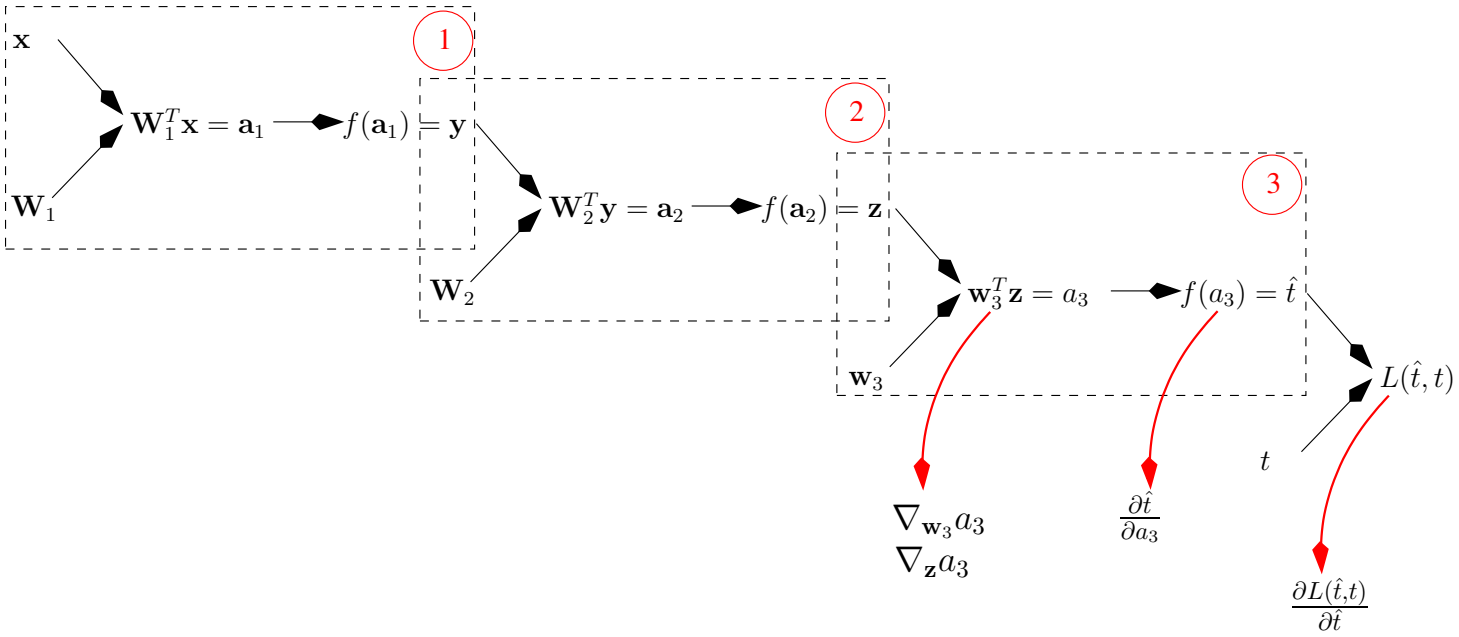
Three Layer Network, simplified and its Jacobians

- Each node should know how to calculate its derivative, gradient, or Jacobian, with respect to:
  - its *input*
  - its *parameters*



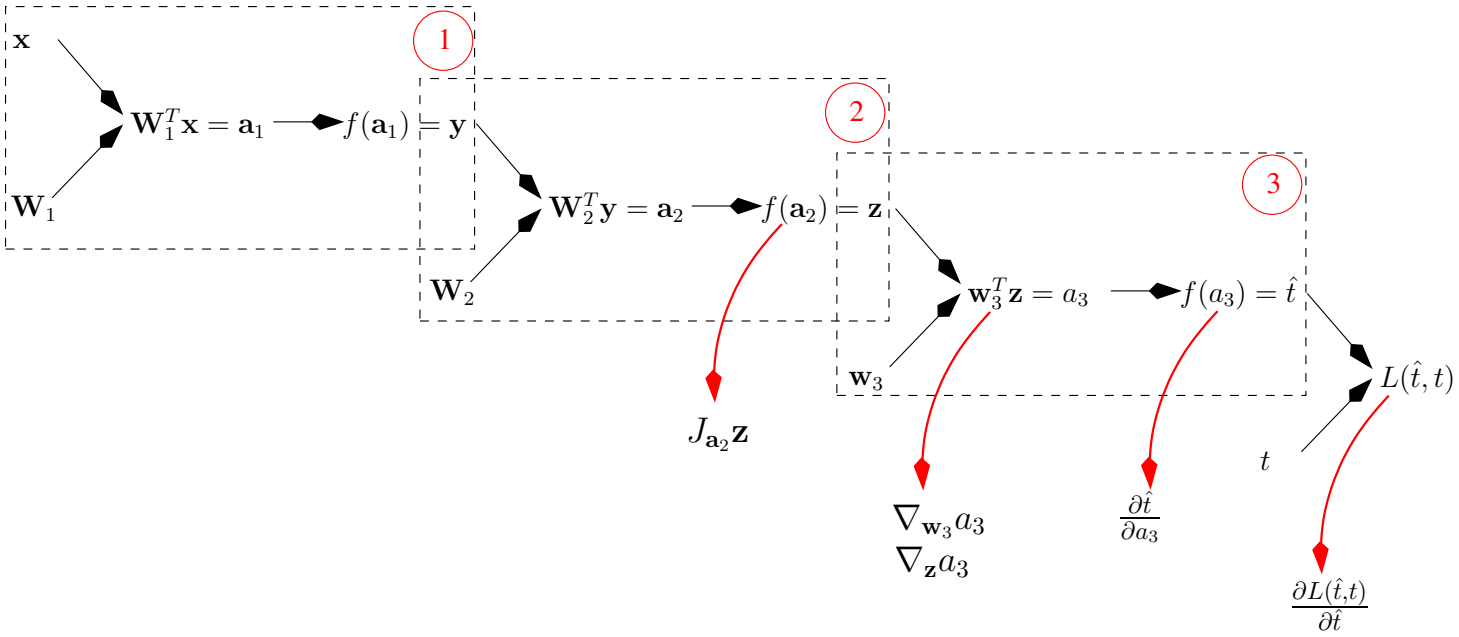
Three Layer Network, simplified and its Jacobians

- Each node should know how to calculate its derivative, gradient, or Jacobian, with respect to:
  - its *input*
  - its *parameters*



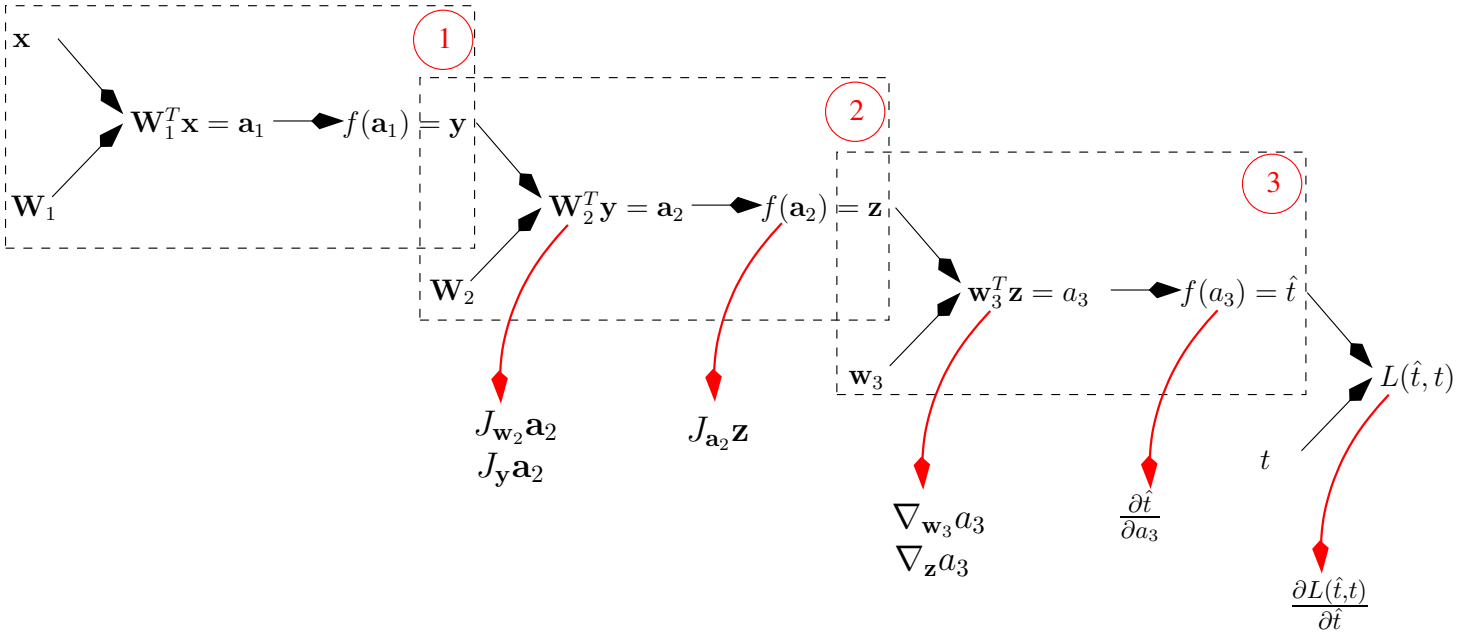
Three Layer Network, simplified and its Jacobians

- Each node should know how to calculate its derivative, gradient, or Jacobian, with respect to:
  - its *input*
  - its *parameters*



Three Layer Network, simplified and its Jacobians

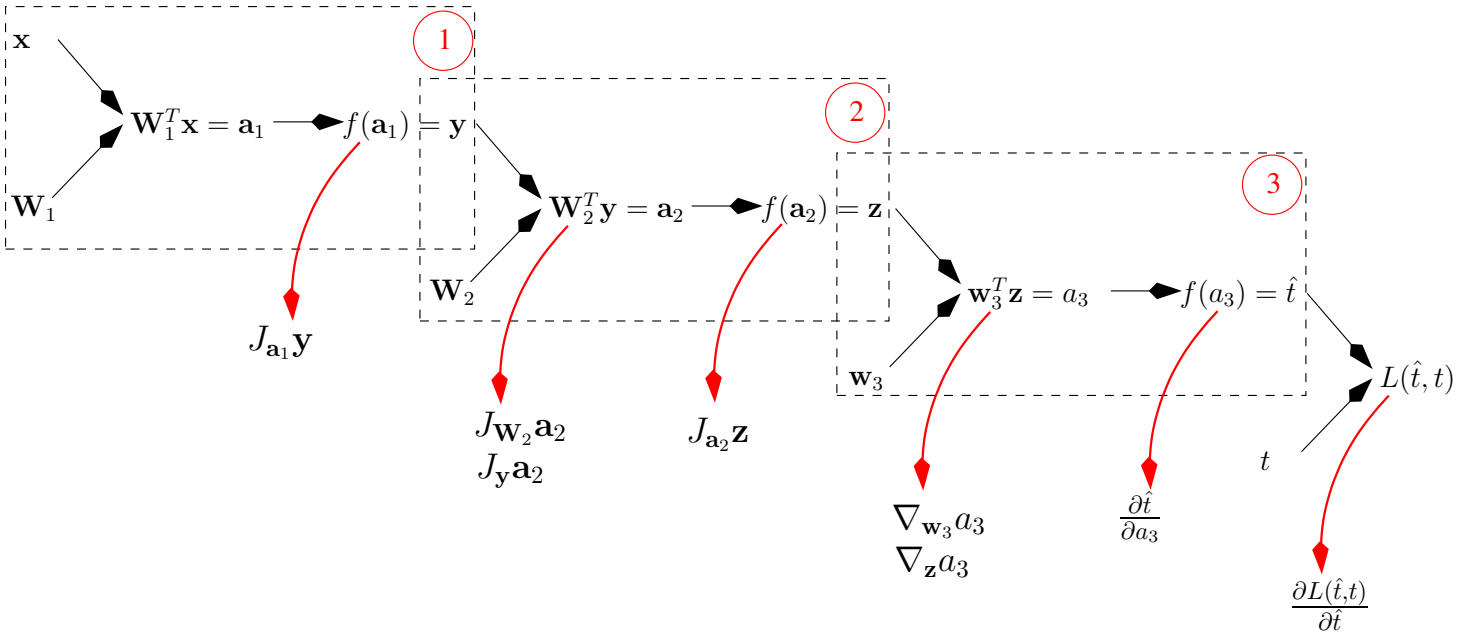
- Each node should know how to calculate its derivative, gradient, or Jacobian, with respect to:
  - its *input*
  - its *parameters*



- The (k, l) item of the $J_{\mathbf{W}_2}\mathbf{a}_2$ is the *vector*: $\nabla_{\mathbf{W}_{2_l}} a_{2_k}$
- Neverthless the $J_{\mathbf{W}_2}\mathbf{a}_2$ matrix is diagonal, since only $\mathbf{W}_{2_k}$ contributes to a_{2_k}

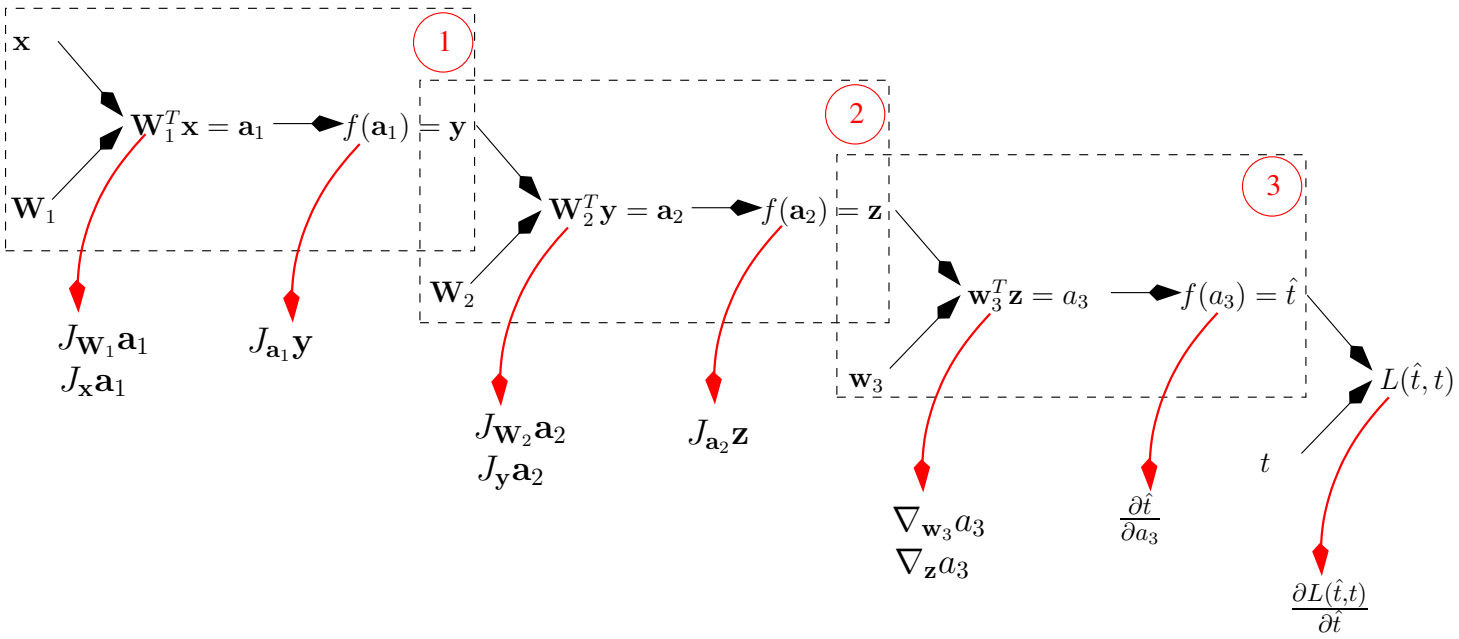
Three Layer Network, simplified and its Jacobians

- Each node should know how to calculate its derivative, gradient, or Jacobian, with respect to:
  - its *input*
  - its *parameters*



Three Layer Network, simplified and its Jacobians

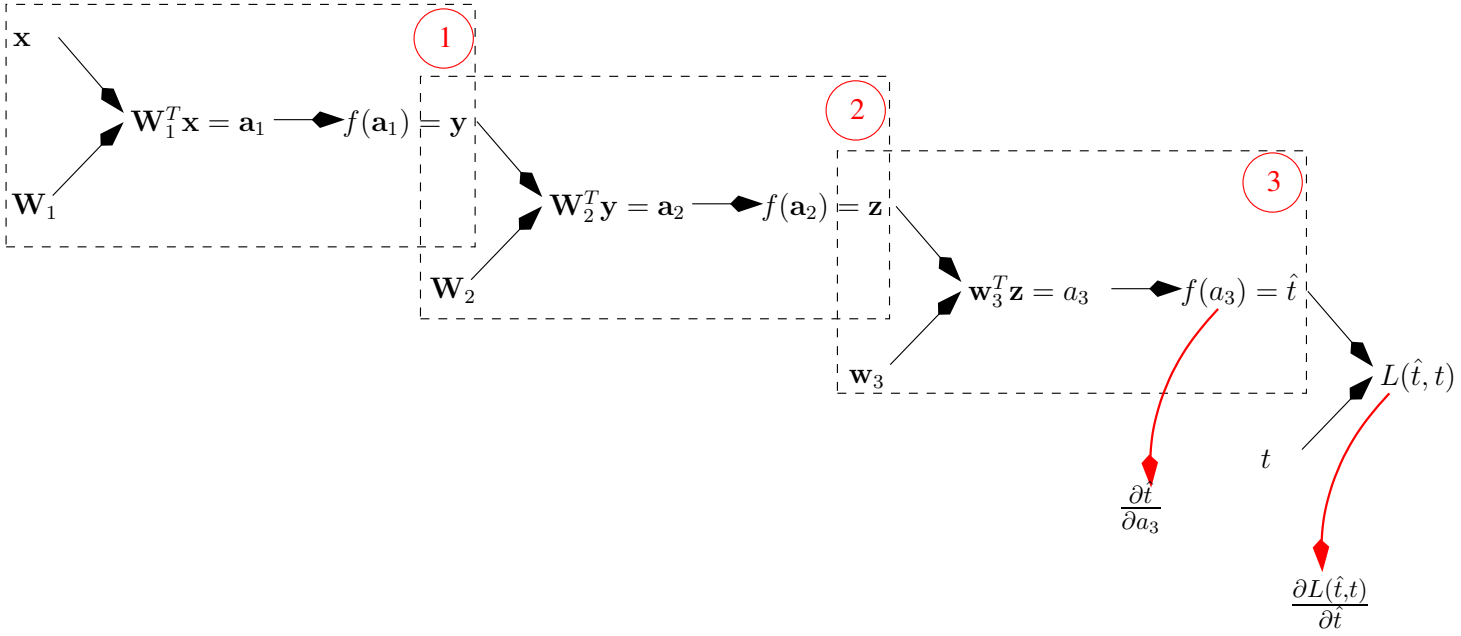
- Each node should know how to calculate its derivative, gradient, or Jacobian, with respect to:
  - its *input*
  - its *parameters*



As before the (k, l) item of the $J_{\mathbf{W}_1}\mathbf{a}_1$ is the *vector*: $\nabla_{\mathbf{W}_{1_l}} a_{1_k}$

Three Layer Network, simplified and its Jacobians

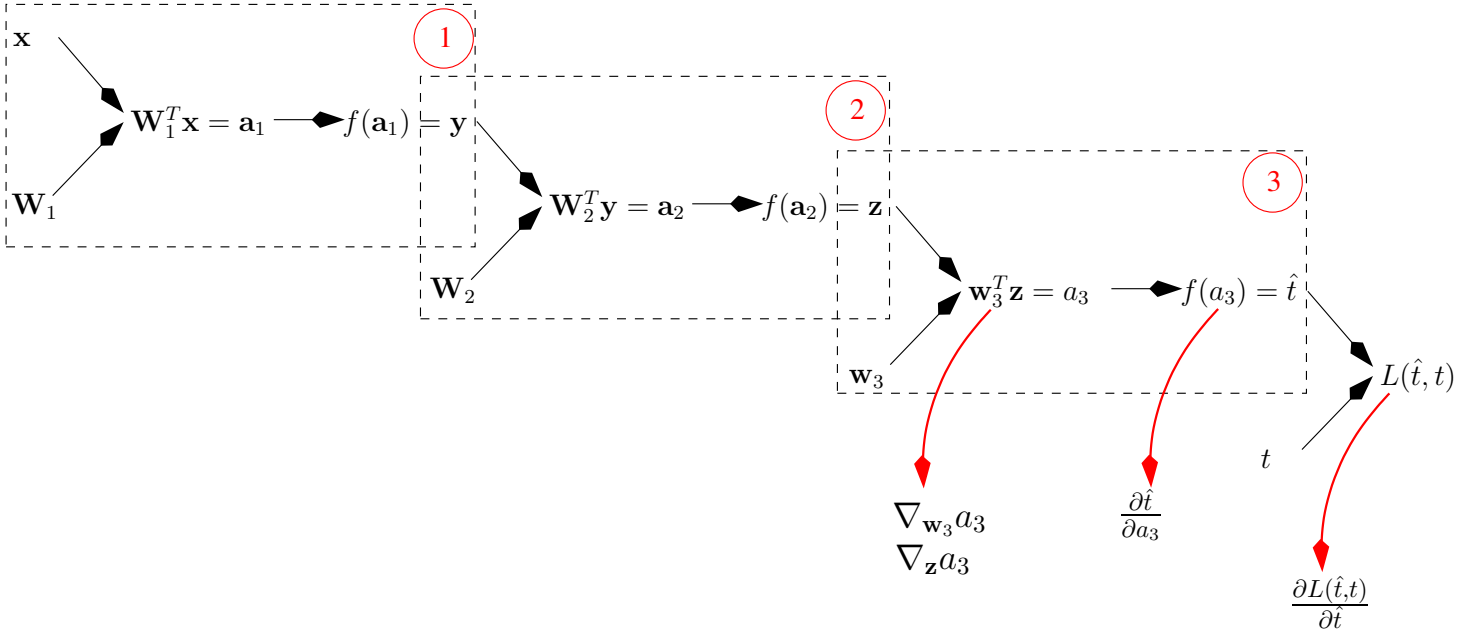
- Then derivative calculation is just chain rule over and over again (*backpropagation*).
  - its *input*
  - its *parameters*



$$\frac{\partial L(\hat{t},t)}{\partial a_3} = \frac{\partial L(\hat{t},t)}{\partial \hat{t}} \frac{\partial \hat{t}}{\partial a_3}$$

Three Layer Network, simplified and its Jacobians

- Then derivative calculation is just chain rule over and over again (*backpropagation*).
  - its *input*
  - its *parameters*

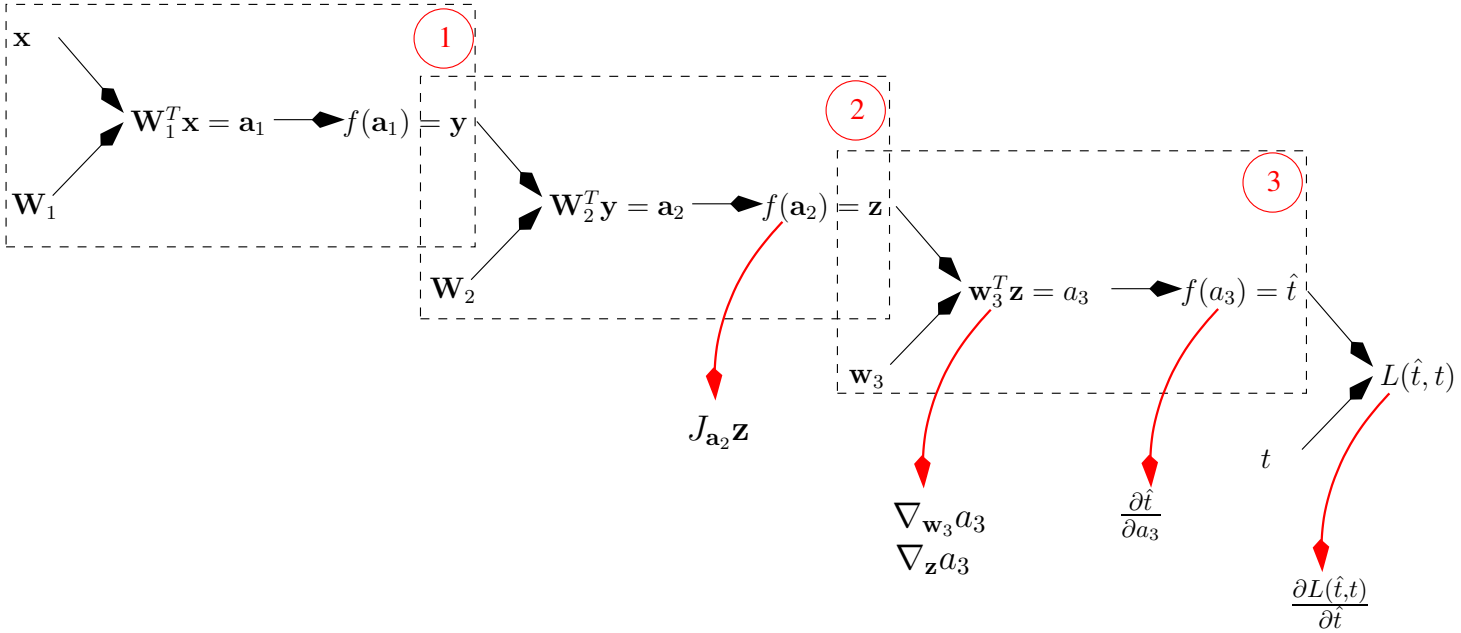


$$\nabla_{\mathbf{w}_3} L(\hat{t}, t) = \frac{\partial L(\hat{t},t)}{\partial \hat{t}} \frac{\partial \hat{t}}{\partial a_3} \nabla_{\mathbf{w}_3} a_3$$

$$\nabla_{\mathbf{z}} L(\hat{t}, t) = \frac{\partial L(\hat{t},t)}{\partial \hat{t}} \frac{\partial \hat{t}}{\partial a_3} \nabla_{\mathbf{z}} a_3$$

Three Layer Network, simplified and its Jacobians

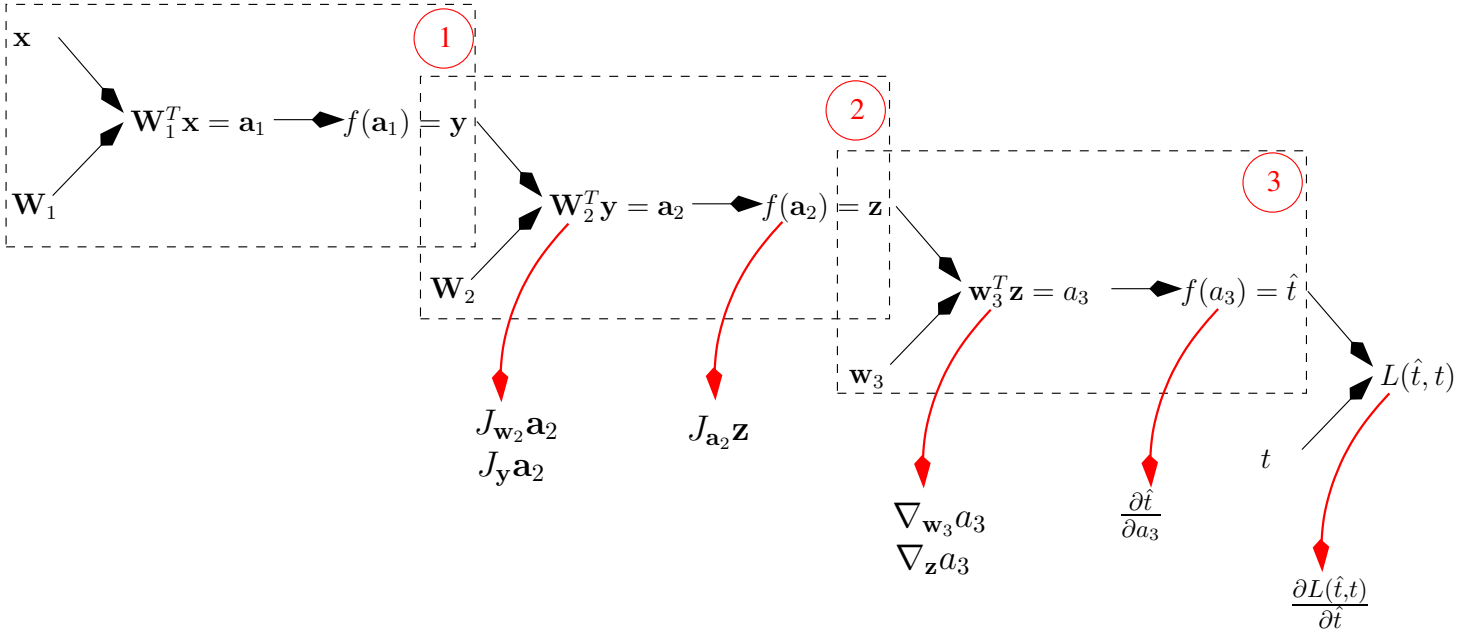
- Then derivative calculation is just chain rule over and over again (*backpropagation*).
  - its *input*
  - its *parameters*



$$\nabla_{\mathbf{a}_2} L(\hat{t}, t) = \frac{\partial L(\hat{t},t)}{\partial \hat{t}} \frac{\partial \hat{t}}{\partial a_3} \nabla_{\mathbf{z}} a_3 J_{\mathbf{a}_2}\mathbf{z}$$

Three Layer Network, simplified and its Jacobians

- Then derivative calculation is just chain rule over and over again (*backpropagation*).
  - its *input*
  - its *parameters*

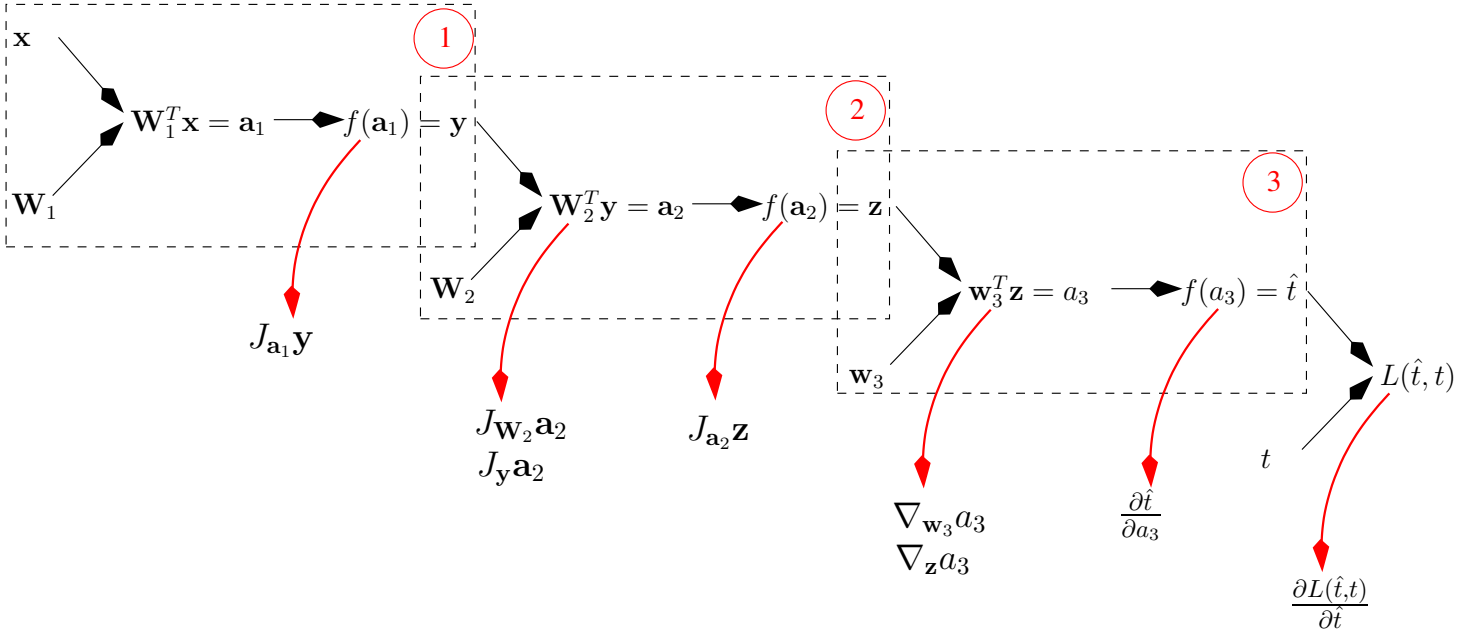


$$\nabla_{\mathbf{W}_2} L(\hat{t}, t) = \frac{\partial L(\hat{t},t)}{\partial \hat{t}} \frac{\partial \hat{t}}{\partial a_3} \nabla_{\mathbf{z}} a_3 J_{\mathbf{a}_2}\mathbf{z} J_{\mathbf{W}_2}\mathbf{a}_2$$

$$\nabla_{\mathbf{y}} L(\hat{t}, t) = \frac{\partial L(\hat{t},t)}{\partial \hat{t}} \frac{\partial \hat{t}}{\partial a_3} \nabla_{\mathbf{z}} a_3 J_{\mathbf{a}_2}\mathbf{z} J_{\mathbf{y}}\mathbf{a}_2$$

Three Layer Network, simplified and its Jacobians

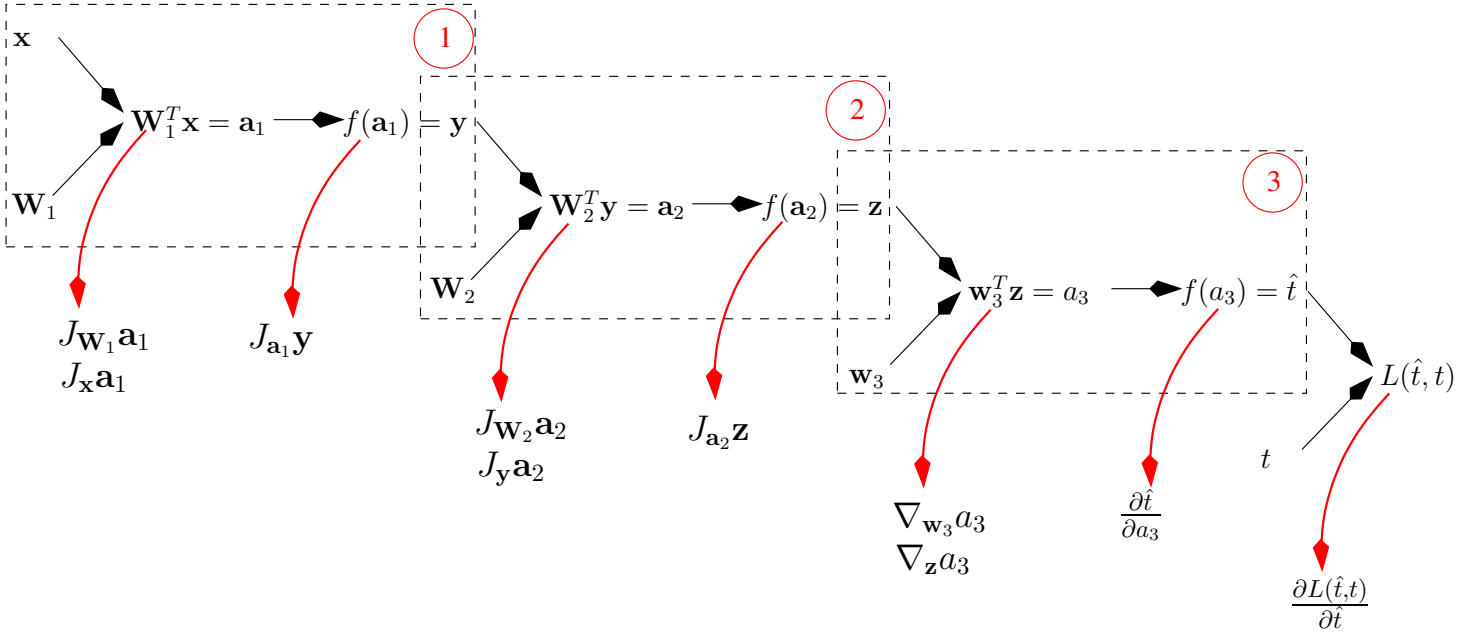
- Then derivative calculation is just chain rule over and over again (*backpropagation*).
  - its *input*
  - its *parameters*



$$\nabla_{\mathbf{a}_1} L(\hat{t}, t) = \frac{\partial L(\hat{t},t)}{\partial \hat{t}} \frac{\partial \hat{t}}{\partial a_3} \nabla_{\mathbf{z}} a_3 J_{\mathbf{a}_2}\mathbf{z} J_{\mathbf{y}}\mathbf{a}_2 J_{\mathbf{a}_1}\mathbf{y}$$

Three Layer Network, simplified and its Jacobians

- Then derivative calculation is just chain rule over and over again (*backpropagation*).
  - its *input*
  - its *parameters*



$$\nabla_{\mathbf{W}_1} L(\hat{t}, t) = \frac{\partial L(\hat{t},t)}{\partial \hat{t}} \frac{\partial \hat{t}}{\partial a_3} \nabla_{\mathbf{z}} a_3 J_{\mathbf{a}_2}\mathbf{z} J_{\mathbf{y}}\mathbf{a}_2 J_{\mathbf{a}_1}\mathbf{y} J_{\mathbf{W}_1}\mathbf{a}_1$$

$$\nabla_{\mathbf{x}} L(\hat{t}, t) = \frac{\partial L(\hat{t},t)}{\partial \hat{t}} \frac{\partial \hat{t}}{\partial a_3} \nabla_{\mathbf{z}} a_3 J_{\mathbf{a}_2}\mathbf{z} J_{\mathbf{y}}\mathbf{a}_2 J_{\mathbf{a}_1}\mathbf{y} J_{\mathbf{x}}\mathbf{a}_1$$

Three Layer Network, simplified and its Jacobians

$$\frac{\partial L(\hat{t},t)}{\partial a_3} = \frac{\partial L(\hat{t},t)}{\partial \hat{t}}\frac{\partial \hat{t}}{\partial a_3}$$

$$(\nabla_{\mathbf{w}_3} L(\hat{t},t))^T = \boxed{\frac{\partial L(\hat{t},t)}{\partial \hat{t}}\frac{\partial \hat{t}}{\partial a_3}}(\nabla_{\mathbf{w}_3} a_3)^T$$

$$(\nabla_{\mathbf{z}} L(\hat{t},t))^T = \boxed{\frac{\partial L(\hat{t},t)}{\partial \hat{t}}\frac{\partial \hat{t}}{\partial a_3}}(\nabla_{\mathbf{z}} a_3)^T$$

$$(\nabla_{\mathbf{a}_2} L(\hat{t},t))^T = \boxed{\frac{\partial L(\hat{t},t)}{\partial \hat{t}}\frac{\partial \hat{t}}{\partial a_3}(\nabla_{\mathbf{z}} a_3)^T} J_{\mathbf{a}_2}\mathbf{z}$$

$$(\nabla_{\mathbf{W}_2} L(\hat{t},t))^T = \boxed{\frac{\partial L(\hat{t},t)}{\partial \hat{t}}\frac{\partial \hat{t}}{\partial a_3}(\nabla_{\mathbf{z}} a_3)^T J_{\mathbf{a}_2}\mathbf{z}} J_{\mathbf{W}_2}\mathbf{a}_2$$

$$(\nabla_{\mathbf{y}} L(\hat{t},t))^T = \boxed{\frac{\partial L(\hat{t},t)}{\partial \hat{t}}\frac{\partial \hat{t}}{\partial a_3}(\nabla_{\mathbf{z}} a_3)^T J_{\mathbf{a}_2}\mathbf{z}} J_{\mathbf{y}}\mathbf{a}_2$$

$$(\nabla_{\mathbf{a}_1} L(\hat{t},t))^T = \boxed{\frac{\partial L(\hat{t},t)}{\partial \hat{t}}\frac{\partial \hat{t}}{\partial a_3}(\nabla_{\mathbf{z}} a_3)^T J_{\mathbf{a}_2}\mathbf{z} J_{\mathbf{y}}\mathbf{a}_2} J_{\mathbf{a}_1}\mathbf{y}$$

$$(\nabla_{\mathbf{W}_1} L(\hat{t},t))^T = \boxed{\frac{\partial L(\hat{t},t)}{\partial \hat{t}}\frac{\partial \hat{t}}{\partial a_3}(\nabla_{\mathbf{z}} a_3)^T J_{\mathbf{a}_2}\mathbf{z} J_{\mathbf{y}}\mathbf{a}_2 J_{\mathbf{a}_1}\mathbf{y}} J_{\mathbf{W}_1}\mathbf{a}_1$$

$$(\nabla_{\mathbf{x}} L(\hat{t},t))^T = \boxed{\frac{\partial L(\hat{t},t)}{\partial \hat{t}}\frac{\partial \hat{t}}{\partial a_3}(\nabla_{\mathbf{z}} a_3)^T J_{\mathbf{a}_2}\mathbf{z} J_{\mathbf{y}}\mathbf{a}_2 J_{\mathbf{a}_1}\mathbf{y}} J_{\mathbf{x}}\mathbf{a}_1$$

Linking the two derivations

- Lets see how we can rewrite the δ-based derivation to a matrix form.
- I will be a bit loose on the notation when it comes to gradients, vectors, matrices and Jacobians.
- Starting from eq ?? which shows how to get δ_j for the j node

$$\delta_j = \frac{\partial E}{\partial a_j} = \sum_k \frac{\partial E}{\partial a_k}\frac{\partial a_k}{\partial a_j} = \sum_k \delta_k \frac{\partial a_k}{\partial a_j}$$

- We switch to the layer/matrix view and we have for layer p

$$\begin{aligned} \mathbf{a_p} &= [a_{p1}, \ldots, a_{pk}]^T \\ \boldsymbol{\delta}_p &= [\delta_{p1}, \ldots, \delta_{pk}]^T \end{aligned}$$

- we are going to compute:

$$\nabla_{\boldsymbol{a}_p} E = [\frac{\partial E}{\partial a_{p1}}, \ldots, \frac{\partial E}{\partial a_{pk}}] = \boldsymbol{\delta}_p$$

Linking the two derivations

Rewritting eq ?? to include the p layer index we have:

$$\delta_{p_j} = \frac{\partial E}{\partial a_{p_j}} = \sum_k \frac{\partial E}{\partial a_{p+1_k}} \frac{\partial a_{p+1_k}}{\partial a_{p_j}} = \sum_k \delta_{p+1_k} \frac{\partial a_{p+1_k}}{\partial a_{p_j}}$$

observing that:

$$\begin{aligned} \nabla_{\mathbf{a_{p+1}}} E &= [\frac{\partial E}{\partial a_{p+1_k}} | k = 1, \ldots, |p+1|]^T \\ J_{\mathbf{a_p}} \mathbf{a_{p+1}}[.\mathbf{j}] &= [\frac{\partial a_{p+1_k}}{\partial a_{p_j}} | k = 1, \ldots, |p+1|] \end{aligned}$$

where [.j] is the jth column of the Jacobian $J_{\mathbf{a_p}} \mathbf{a_{p+1}}$

we get:

$$\begin{aligned} \delta_{p_j} &= (\nabla_{\mathbf{a_{p+1}}} E)(J_{\mathbf{a_p}} \mathbf{a_{p+1}}[\mathbf{j}.]) \Rightarrow \\ \boldsymbol{\delta}_p^T &= (\nabla_{\mathbf{a}_{p+1}} E)^T J_{\mathbf{a_p}} \mathbf{a_{p+1}} = \boldsymbol{\delta}_{p+1} J_{\mathbf{a_p}} \mathbf{a_{p+1}} \end{aligned}$$

Linking the two derivations

We have:

$$\mathbf{a}_p = \mathbf{W}_p^T \mathbf{z}_{p-1}$$

Thus:

$$\begin{aligned}(\nabla_{\mathbf{W}_p} E)^T &= \nabla_{\mathbf{a}_p} E \mathbf{J}_{\mathbf{W}_p} \mathbf{a}_p \\ &= \boldsymbol{\delta}_p \mathbf{J}_{\mathbf{W}_p} \mathbf{a}_p\end{aligned}$$

where the Jacobian $\mathbf{J}_{\mathbf{W}_p} \mathbf{a}_p$ is given by:

$$\mathbf{J}_{\mathbf{W}_p} \mathbf{a}_p = \begin{bmatrix} \nabla_{\mathbf{W}_{p1}} a_{p1} & \nabla_{\mathbf{W}_{p2}} a_{p1} & \dots & \nabla_{\mathbf{W}_{pk}} a_{p1} \\ \nabla_{\mathbf{W}_{p1}} a_{p2} & \nabla_{\mathbf{W}_{p2}} a_{p2} & \dots & \nabla_{\mathbf{W}_{pk}} a_{p2} \\ \dots & & & \\ \nabla_{\mathbf{W}_{p1}} a_{pk} & \nabla_{\mathbf{W}_{p2}} a_{pk} & \dots & \nabla_{\mathbf{W}_{pk}} a_{pk} \end{bmatrix}$$

Note that this is a diagonal table since $\mathbf{W}_{pl}$ only contributes to a_{pl}